PITCH DECK · MADE TO MOVE

SOROPROVE

Friendly Lean proofs for Soroban contracts, without any of the sorrow

 Renato **Marziano** &  Julian **Konchunas**

Hack Meridian / 2026 LISBON | GUAVA® Stellar

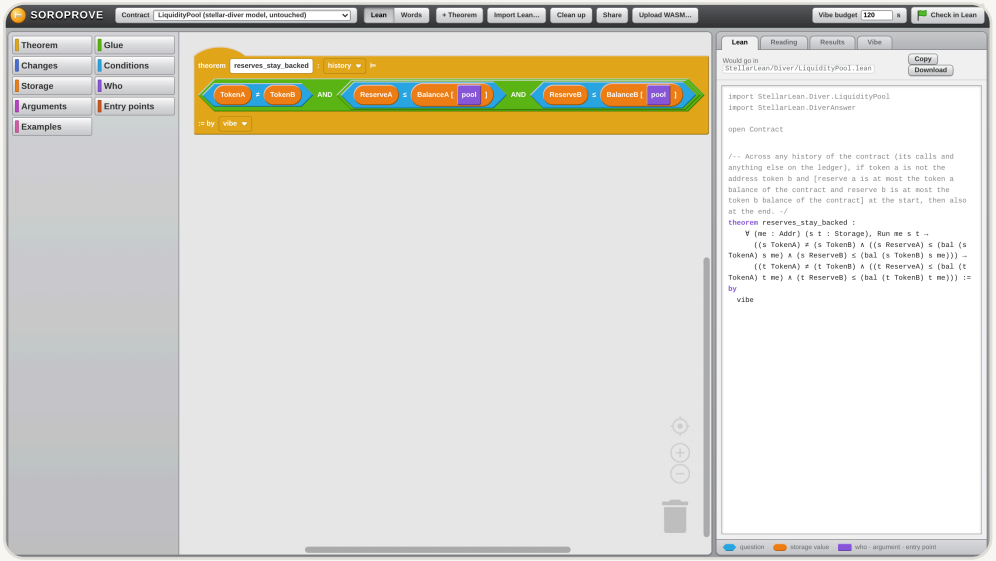
© 2026 HackMeridian · Lisbon

PITCH STEP 01 / 04

THE SOLUTION

Soroprove turns WASM contracts into Lean object representations

- abstracting away every contract code path leading to value transfers into a state machine
- modelling part of the Soroban runtime in Lean
- a higher level UI, automatically generated from stellar contract builds
- a lower level Rust macro integration, which allows for custom invariant proofs



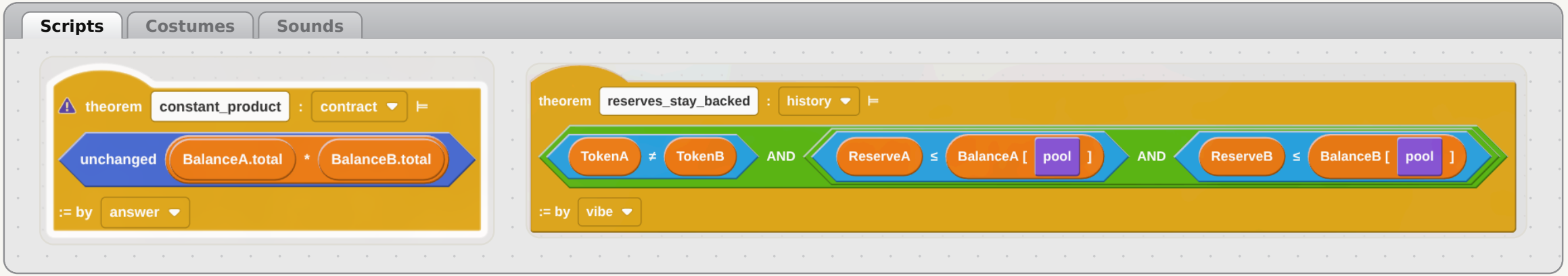
We love Scratch. **Everybody loves Scratch.**

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 01 / 04

INVARIANTS IN BLOCKS

Drop a WASM, start laying out desirable invariants and see if they hold.



Suppose that we wanna prove the reserve solvency of an AMM: pretty simple to express in Soroprove, and checked automatically in a very fast manner.

permalink here

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 02 / 04

PMF

Formal verification is hard, yet it's ever more crucial to constraint complex financial codebases, which are now largerly written by AI.

- Back in August, the Comet protocol was hacked: `execute_swap_exact_amount_in` accepts swaps from the same token onto the same token, leading to broken accounting.
- Some proofs can be quite complex and not obvious, and this is usually the boundary where SMT provers stop.
- Transpiling the entire contract to Lean requires devs to be very skilled in Lean.

We're interested about **fully securing a protocol**, and not finding particular bugs.

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 03 / 04

TECHNICAL WORKFLOW

Soroprove operates directly on the contract's compiled WASM

Only a few syscalls that we care about recreating and tracing through:

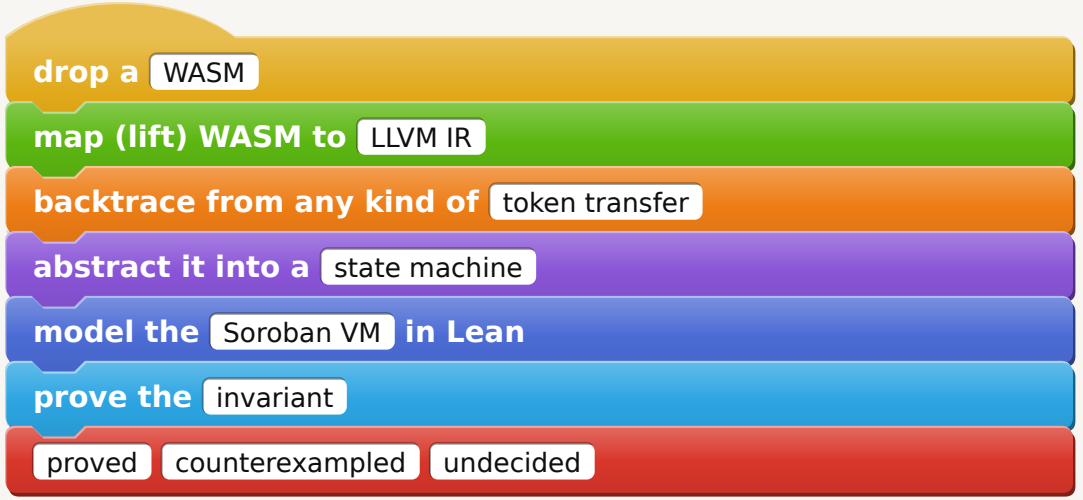
`call` `try_call` value transfers

`require_auth` `require_auth_for_args` permission gates

`update_current_contract_wasm` upgrades

`put_contract_data` `get_contract_data`

`has_contract_data` `del_contract_data` side effects



© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 03 / 04

SOUND BUT NOT COMPLETE

We aim for a final model that is sound but not complete

- everything that we prove on the model will necessarily also hold in the contract
- disproving something doesn't necessarily mean that we found a bug in our contract

Essentially, we derive an **over-approximation** of our contract that we can easily reason and prove properties on.

We can easily match SEP-41 signatures, for example `transfer`:

```
call(token_addr,
     Symbol("transfer"),
     Vec[ from   : AddressObject,
          to     : AddressObject,
          amount: I128Small || I128Object])
```

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 03 / 04

PROOFS BY VIBE

We just lock an LLM in a sandbox and bother it until we either find a proof, or we run out of time

agent edits the proof body, runs ./check

each version is checked by the server, not by the agent

reject sorry native_decide new axioms #eval

proved counterexampled undecided

```
13
14 def cexStep0 : Step := { call := cexCall, pre := ce
15
16 def swap_books_track_input_cex : Step :=
17   { call := cexCall, pre := cexPre,
18     post := swap_exact_amount_in.s3 cexStep0 cexU,
19     writes := [.bal 3 4, .bal 3 2, .bal 3 2, .bal 3
20       AllRecordData.scalar 3, AllRecordData.weight
21       AllRecordData.index 3, AllRecordData.scalar 3,
22     ret := 0 }
23
24 theorem swap_books_track_input_cex_breaks : (∃ e ∈ e
25   refine ⟨(swap_exact_amount_in, by simp [entries],
26   · constructor <;> simp [swap_books_track_input_ce
27   · simp [swap_books_track_input_cex, cexStep0, cex(
28
29 theorem swap_books_track_input_false : ¬ (∀ st, (∃ e
30       (((st.call.fn = "swap_exact_amount_in") ∧ (st
31   swap_books_track_input_cex_breaks.2 (h _ swap_bool
```

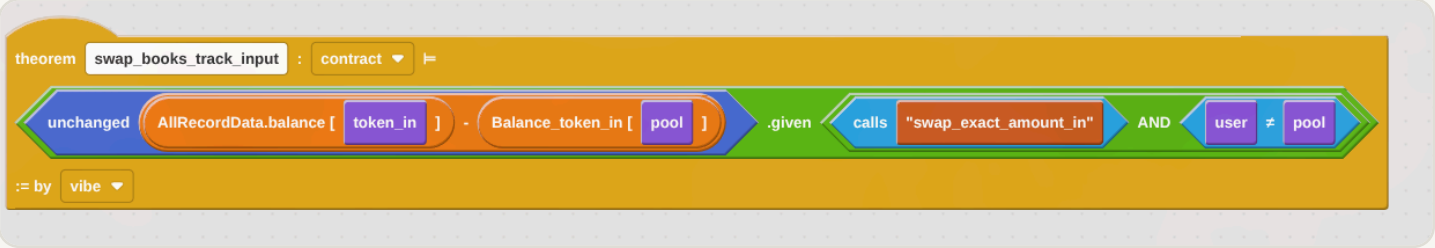
▼ agent (10)
0s agent started
3.9s Bash Comet.lean | head
9.9s Bash Comet.lean; cat Proof.lean check
25s Bash check
35.3s Write Proof.lean
35.8s Bash check
46.8s Bash check
50.9s Fixing a parse error in the counterexample definitions, then rechecking.

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 03 / 04

A FINAL EXAMPLE

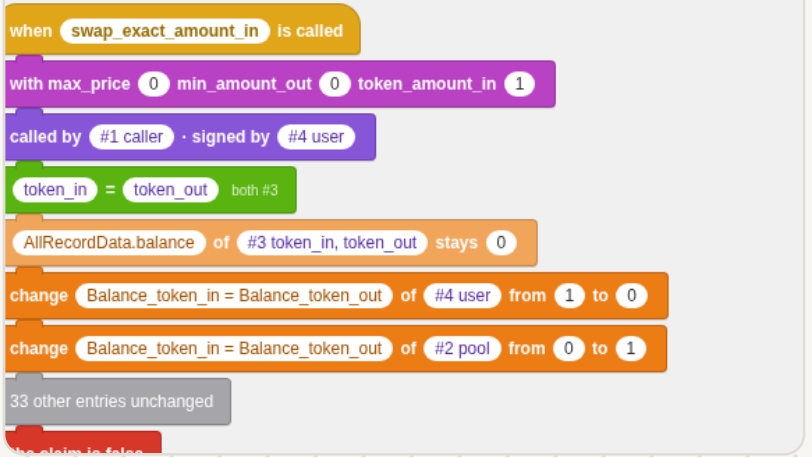
Can we find the counterexample that led to the Comet hack by trying to prove book–balance consistency?



No constraint in the system prevents us from setting `token_in = token_out`, which is exactly what happens in the counterexample.

```
∀ st, (∃ e ∈ entries, e st) →
    (((st.call.fn = "swap_exact_amount_in") ∧ (st.call.argA "user") ≠ (pool st)) →
(((st.post (AllRecordData.balance (st.call.argA "token_in"))) - (bal (st.call.argA
"token_in") st.post (pool st))) = ((st.pre (AllRecordData.balance (st.call.argA
"token_in"))) - (bal (st.call.argA "token_in") st.pre (pool st)))))
```

```
1 /-- A self-swap: token_in = token_out = 3. -/
2 def swap_books_track_input_cex : Step := { call := { fn := "swap_exact_amount_in",
3     arg := fun n => if n = "token_amount_in" then 1 else 0, signers := [4] }, pre :
4 theorem swap_books_track_input_false : ¬ (∀ st, (∃ e ∈ entries, e st) → True) := by
5   intro h
6   have := h swap_books_track_input_cex (_, by simp [entries], rfl)
7   simp at this -- the book stays 0, the pool's balance goes 0 → 1
8   omega
```



© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 04 / 04

THE TEAM

Scripts | Costumes | Sounds



Renato

when clicked
say Renato Marziano
Security Researcher & Auditor @ OtterSec
CTF player · reverse engineering



Julian

when clicked
say Julian Konchunas
Security Auditor @ OtterSec
hacker

We're experienced auditors and hackers, the right people to make formal verification friendly.

© 2026 HackMeridian · Lisbon · #HackMeridian26